

OPEN SOURCE LEARNING PLATFORM

reQuest

학습 기록과 오답을 맞춤형 복습 퀘스트로 연결하는
오픈소스 자기주도 학습 플랫폼

상세 프로젝트 계획서

프로젝트명	reQuest
프로젝트 유형	교육·학습 / 오픈소스 / 웹 서비스
개발 형태	3 인 팀 프로젝트
팀 구성	백진욱(팀장) · 김하늘 · 장근우
문서 버전	상세 계획서 v3.0 / 2026.07.30

"체크해서 끝나는 플래너가 아니라, 이해해야 끝나는 플래너"

목차

1. 프로젝트 개요	11. AI 문제 생성 설계
2. 추진 배경 및 문제 정의	12. 화면 구성
3. 프로젝트 목표	13. 오픈소스 활용 및 공개 전략
4. 핵심 사용자 및 사용 시나리오	14. 개발 일정 및 역할 분담
5. 핵심 기능 설계	15. 테스트 및 품질 관리
6. 서비스 동작 흐름	16. 위험요소 및 대응
7. 차별성 및 경쟁력	17. 기대 효과
8. MVP 범위와 단계별 확장	18. 향후 발전 방향
9. 시스템 구성 및 기술 스택	19. 최종 완료 기준
10. 데이터베이스 설계	

문서 개정 핵심

v3.0 개정 사항

현재 완성된 프론트엔드 MVP 와 향후 개발할 백엔드·PostgreSQL·AI 연동을 분리해 표시하고, 익명 역할명을 제거하여 백진욱·김하늘·장근우의 실제 담당 업무와 완료 기준을 명확히 정리했다.

구분	현재 상태	다음 단계
프론트엔드	주요 사용자 흐름 MVP 구현 완료	반응형·접근성·예외 상태 보완
데이터	브라우저 localStorage 기반	PostgreSQL 스키마와 마이그레이션 적용
문제 생성	규칙 기반 Mock Generator	교체 가능한 AI Adapter 연결
협업	GitHub 저장소 개설·main 업로드	브랜치·PR·이슈 기반 분업

1. 프로젝트 개요

1.1 프로젝트명과 의미

reQuest 는 Review(복습)와 Quest(임무)를 결합한 이름이다. 사용자가 공부한 내용과 오답을 기록하면 시스템이 복습 문제를 구성하고, 핵심 개념을 확인한 뒤 해당 날짜의 학습을 완료하는 구조를 의미한다.

한 줄 소개

학습 기록과 오답을 기반으로 맞춤형 복습 문제를 만들고, 풀이 결과로 이해도를 확인하는
오픈소스 자기주도 학습 플랫폼

1.2 프로젝트 범위

- 주간 학습 계획과 일별 학습 항목 관리
- 학습 기록 작성·조회·검색·삭제
- 오답 문제 이미지 및 텍스트 등록과 상세 보기
- 객관식·OX·단답형 복습 문제 검토, 수정, 승인
- 퀴즈 풀이, 즉시 채점, 해설, 재시도 상태 관리
- 학습시간·이해도·활동일·연속 기록·히트맵 통계
- 향후 AI 기반 취약 개념 분석과 변형 문제 생성

2. 추진 배경 및 문제 정의

2.1 기존 학습 플래너의 한계

일반 To-do List 와 공부 플래너는 계획 등록과 완료 체크에 집중한다. 사용자가 특정 단원을 완료했다고 표시해도 내용을 실제로 이해했는지, 시간이 지난 뒤에도 기억하는지는 확인하기 어렵다.

- 완료 여부가 사용자의 주관적인 체크에 의존한다.
- 학습시간과 실제 이해도가 분리되어 기록된다.
- 틀린 문제의 반복 복습 시점이 관리되지 않는다.
- 오답노트 작성·분류·재출제의 부담이 크다.
- 과목별 취약 개념이 지속적으로 누적되지 않는다.

2.2 해결하려는 핵심 문제

문제 정의

학습 계획과 실제 이해도 검증이 분리되어 사용자가 취약 개념을 놓친 채 학습을 완료했다고 판단하는 문제

3. 프로젝트 목표

목표 영역	구체적 목표
기능	학습 기록부터 오답 등록, 문제 검토, 퀴즈, 통계까지 하나의 흐름으로 연결
교육	회상 연습과 반복 복습을 통해 취약 개념 중심의 자기주도 학습 루프 형성
오픈소스	문제 생성 스키마, 완료 판정 규칙, AI Adapter 구조와 실행 문서 공개
품질	핵심 흐름을 5 분 안에 안정적으로 시연하고, 오류 시 사용자의 기록을 보호

3.1 성공 지표

지표	목표
사용성	처음 사용하는 사람이 도움 없이 5 분 안에 핵심 흐름 완료
기능 안정성	학습 기록부터 퀴즈 결과까지 데이터 손실 없이 연결
문제 품질	정답·해설·유형이 검증된 문항만 퀴즈에 포함
재현성	README 와 예시 설정만으로 팀원 환경에서 실행 가능

4. 핵심 사용자 및 사용 시나리오

4.1 주요 사용자

사용자	주요 문제	reQuest 활용 방식
대학생	전공·자격증 학습 후 복습 누락	학습 기록과 오답을 당일 복습 퀘스트로 전환
자격증 준비생	암기량이 많고 취약 개념 반복 필요	개념형 문제와 재시도로 반복 확인
중·고등학생	계획은 세우지만 이해도 확인 부족	문제 사진과 학습 내용을 등록해 복습
교육 서비스 개발자	퀘스트형 복습 기능을 직접 구축해야 함	공개 스키마·모듈·API 구조 재사용

4.2 대표 사용 시나리오

1. 사용자가 과목, 학습 제목, 공부한 내용과 학습 시간을 기록한다.
2. 틀린 문제의 이미지와 정답, 자신이 고른 답, 틀린 이유를 오답노트에 등록한다.
3. 시스템이 학습 내용과 오답을 바탕으로 복습 문제 후보를 구성한다.
4. 사용자가 문제·정답·해설을 검토하고 필요한 부분을 수정한 뒤 승인한다.
5. 객관식·OX·단답형 퀴즈를 풀고 즉시 채점과 해설을 확인한다.
6. 풀이 결과와 이해도가 학습 기록과 통계에 반영된다.

4.3 지원 과목 범위

텍스트로 개념과 정답을 표현할 수 있는 교과·전공·자격증 과목을 우선 지원한다. 국어, 영어, 사회, 역사, 과학 개념, 컴퓨터공학, 자격증 이론 학습 등에 적합하다. 복잡한 수식 인식, 손글씨 OCR, 도형 중심 문제는 2차 확장 범위로 둔다.

5. 핵심 기능 설계

기능	동작	현재 상태
이번 주 학습 보드	주간 이동, 오늘 표시, 일별 항목과 완료 수 확인	완료
학습 기록	과목·제목·내용·시간·이해도·키워드 저장 및 조회	완료
오답노트	이미지·문제·정답·오답·이유·개념 태그 등록	완료
문제 검토	문제 유형·보기·정답·해설 수정 후 승인	완료(Mock)
퀴즈	한 문제씩 풀이, 즉시 채점, 해설, 점수 표시	완료(Mock)
통계	14 일 학습시간, 이해도 변화, 활동일, 히트맵	완료
서버 저장	계정별 기록과 이미지, 퀴즈 결과 영구 저장	개발 예정
AI 복습	취약 개념 추출과 변형 문제 자동 생성	개발 예정

5.1 퀘스트 완료 판정

완료 기준

QuestCompleted = 모든 필수 문항 제출 AND 미해결 문항 0 개. AI 연동 이후에는 핵심 개념별 통과 여부까지 포함한다.

6. 서비스 동작 흐름

01	02	03	04	05	06
학습 기록	오답 등록	문제 생성	검토·승인	퀴즈 풀이	통계 반영

6.1 데이터 처리 흐름

현재 MVP 는 브라우저 localStorage 에 학습 기록, 오답노트, 검토한 퀘스트와 풀이 이력을 저장한다. 백엔드 연결 후에는 입력 검증 → REST API → PostgreSQL 저장 → 문제 생성 서비스 → 사용자 승인 → 채점·통계 갱신 순서로 처리한다.

7. 차별성 및 경쟁력

구분	일반 To-do	일반 오답노트	reQuest
완료 기준	사용자가 체크	복습 여부 불명확	풀이 결과와 개념 통과
오답 관리	메모 수준	사용자가 직접 정리	이미지·텍스트 기반 구조화
복습 방식	일정 재등록	원문 재풀이	유형을 바꾼 퀘스트
학습 검증	없음	사용자 자율	즉시 채점과 이해도 기록
데이터	일정 중심	문제 보관 중심	기록·오답·퀴즈·통계 연결
오픈소스	낮음	낮음	생성·판정 모듈 재사용

핵심 차별 문장

공부한 시간을 기록하는 서비스가 아니라, 학습한 내용을 실제로 이해했는지 확인하여
완료를 결정하는 서비스

8. MVP 범위와 단계별 확장

8.1 현재 완료된 프론트엔드 MVP

- 주간 학습 보드와 오늘 날짜 자동 표시
- 학습 기록 작성·목록·상세 보기·삭제
- 오답노트 작성·목록·상세 보기·검색·필터
- 오답 이미지 선택·파일명·미리보기·삭제
- 복습 문제 검토·수정·승인
- 객관식·OX·단답형 퀴즈와 즉시 채점
- 학습시간·이해도 변화·활동일·히트맵 통계
- 빈 상태·복습 대기·마스터 상태 표현

8.2 MVP 이후 우선 개발

- Express API 와 PostgreSQL 연결
- 학습 기록·오답노트·퀴즈 결과 CRUD API
- 이미지 업로드와 파일 형식·용량 검증
- Mock Generator 를 AI Adapter 로 교체
- 프론트 localStorage 를 서버 API 호출로 단계적 전환
- 통합 테스트, 배포, README 와 시연 영상 완성

8.3 제출 범위에서 제외

손글씨·수학식 완전 자동 OCR, 교사용 관리 페이지, 친구 대결·채팅, 모바일 네이티브 앱, 복잡한 머신러닝 추천은 핵심 흐름의 완성도를 위해 제출 MVP 에서 제외한다.

9. 시스템 구성 및 기술 스택

9.1 기술 스택

영역	기술·도구	역할
개발 환경	VS Code	프론트엔드·백엔드 공통 개발 환경
프론트엔드	React 19, TypeScript, Vite	화면 구성, 상태 관리, 사용자 상호작용
라우팅	React Router	페이지 이동과 상세 경로 관리
통계	Recharts	막대·선 그래프와 툴팁, 학습 히트맵
UI	Lucide React, 페이지별 CSS	아이콘과 reQuest 디자인 시스템
백엔드	Node.js, Express, TypeScript	REST API, 검증, 채점, 상태 관리
데이터베이스	PostgreSQL, pgAdmin	계정·기록·오답·문제·풀이 결과 저장
업로드	Multer	오답 이미지 업로드 처리
검증	Zod	요청 데이터와 AI JSON 스키마 검증
협업	Git, GitHub, Postman	버전 관리, 코드 리뷰, API 테스트

9.2 시스템 아키텍처

구성

React Web Client → Express REST API → Service

Layer(Record·WrongNote·Quest·Scoring) → PostgreSQL / File Storage /

QuestionGenerator Adapter(Mock 또는 AI Provider)

10. 데이터베이스 설계

실제 PostgreSQL 서버나 비밀번호는 GitHub 에 올리지 않는다. 저장소에는 database/schema.sql, database/seed.sql, database/README.md 와 .env.example 을 공개해 팀원이 동일한 구조를 재현하도록 한다.

테이블	주요 필드	용도
users	id, nickname, email, created_at	사용자 기본 정보
subjects	id, user_id, name, color	사용자별 과목
study_records	id, subject_id, title, content, duration, understanding, studied_at	학습 기록
wrong_notes	id, study_record_id, image_path, question, correct_answer, wrong_answer, reason	오답 기록
concepts	id, name, description	개념 사전
weak_concepts	id, wrong_note_id, concept_id, confidence	취약 개념 연결
review_requests	id, source_type, source_id, status, approved_at	검토·승인된 복습 세트
review_questions	id, quest_id, type, body, options, answer, explanation	복습 문항
quiz_attempts	id, quest_id, question_id, answer, is_correct, attempt_no	답안 이력
study_stats	id, user_id, date, total_minutes, accuracy, completed	일별 통계

10.1 데이터 원칙

- 사용자별 데이터 접근 범위를 분리하고 모든 외래키 관계를 명시한다.
- 이미지는 파일 저장소에 두고 데이터베이스에는 경로와 메타데이터만 저장한다.
- 정답·풀이 결과는 통계 재계산이 가능하도록 원본 이력을 보존한다.
- 실제 .env, 비밀번호, 개인정보, PostgreSQL 내부 데이터 폴더는 커밋하지 않는다.

11. AI 문제 생성 설계

11.1 현재와 목표 구조

단계	구현 방식	목적
현재 시연	규칙 기반 Mock Generator	외부 API 없이 전체 사용자 흐름 검증
1 차 연동	QuestionGenerator Adapter + AI API	취약 개념과 문제 후보 자동 생성
안정화	Zod 검증 + 사용자 검토·승인	형식 오류와 잘못된 정답 출제 방지
확장	재출제·간격 반복·난이도 조정	정답 암기보다 개념 회상 강화

11.2 입력·출력 규칙

- 입력: 과목, 학습 내용, 원본 문제, 정답, 사용자가 고른 오답, 틀린 이유, 난이도, 문제 수
- 출력: type, question, options, answer, explanation, conceptTag, difficulty 를 포함한 JSON
- 사용자가 제공한 학습 범위 안에서만 문제를 생성한다.
- 정답이 모호하거나 복수 정답 가능성이 있는 문항은 저장하지 않는다.
- 사용자 승인 전에는 생성 문항을 실제 퀴즈에 출제하지 않는다.

11.3 답 암기 문제에 대한 대응

동일 문항을 반복하기보다 핵심 개념은 유지하면서 보기 순서, 질문 표현, 적용 상황을 바꾼 문제를 생성한다. 풀이 직후뿐 아니라 일정 시간이 지난 뒤 다시 출제해 단순한 정답 문구 암기와 개념 이해를 구분한다.

11.4 오류 대응

- AI 응답 형식 오류 시 재요청 또는 기본 템플릿 문제 제공
- 문제·정답·해설의 사용자 편집과 문항 삭제 지원
- API 장애 시 Mock 문제 또는 기존 승인 문제로 시연
- 서버 로그에는 키·개인정보·원문 이미지를 불필요하게 남기지 않음

12. 화면 구성

화면	핵심 구성	경로
메인/이번 주	주간 이동, 학습 여정, 일별 항목, reQuest Lab, 연속 학습	/
학습 기록 작성	과목·제목·내용·시간·이해도·키워드 입력	/records
학습 기록	검색·필터·통계 요약·기록 카드	/history
기록 상세	학습 내용·오답 이미지·퀘스트 이동	/history/:recordId
오답노트	목록·상태·검색·필터·상세 보기	/wrong-notes
오답 작성	이미지·정답·오답·원인·개념 입력	/wrong-notes/new
문제 검토	유형·보기·정답·해설 수정 후 승인	/quest-review/:recordId
퀴즈	진행률·선택지·채점·해설·결과	/quiz/:recordId
통계	학습시간·이해도 변화·활동일·히트맵	/statistics

12.1 UI 원칙

- 사진 1 번의 따뜻한 종이 질감과 검정·아이보리·금색 포인트를 일관되게 사용한다.
- 페이지별 CSS 파일을 분리해 수정 범위와 충돌을 줄인다.
- 빈 상태, 로딩, 저장 성공, 오류, 복습 대기 상태를 사용자에게 명확히 알린다.
- 모바일·태블릿·데스크톱에서 핵심 기능이 잘리지 않도록 반응형을 적용한다.

13. 오픈소스 활용 및 공개 전략

13.1 공개 저장소 구성

GitHub

<https://github.com/jinwook0308/reQuest>

frontend / backend / database / docs 를 하나의 저장소에서 관리하는 모노레포 구조를 사용한다.

공개 항목	내용
frontend	React 기반 사용자 화면과 페이지별 CSS
backend	Express API, 서비스 계층, 검증·채점 로직
database	PostgreSQL schema.sql, seed.sql, 설정 안내
docs	아키텍처, 문제 JSON 스키마, API 명세, 시연 자료
프로젝트 문서	README, LICENSE, CONTRIBUTING, SECURITY, .env.example

13.2 라이선스·보안 원칙

팀 자체 코드는 MIT 또는 Apache-2.0 중 의존성 검토 후 선택한다. 실제 API 키, 데이터베이스 비밀번호, 사용자 데이터와 저작권이 있는 문제집 원문은 공개하지 않는다. 공개 데이터는 팀이 작성한 예시와 익명화된 테스트 데이터만 사용한다.

14. 개발 일정 및 역할 분담

14.1 단계별 일정

단계	핵심 작업	완료 기준	상태
1 단계	요구사항·디자인·프론트 MVP·GitHub	전체 화면 흐름과 Mock 기능 작동	완료
2 단계	Express 기본 구조·PostgreSQL 설계·연결	/api/health 와 DB 연결 확인	다음 작업
3 단계	학습 기록·오답노트 CRUD·이미지 업로드	프론트 입력이 DB 에 저장·조회	예정
4 단계	문제 생성 Adapter·검토·퀴즈 API	문제 JSON 생성부터 채점까지 연결	예정
5 단계	통계·오답 재출제·완료 판정	핵심 사용자 흐름 E2E 통과	예정
6 단계	반응형·테스트·배포·README·영상	대회 제출물과 시연 안정화	예정

14.2 팀 역할

세 사람은 각자 맡은 산출물의 1 차 책임자가 되며, 주 2 회 통합 점검에서 서로의 작업을 함께 확인한다. 업무 난이도 차이는 페어 작업과 작은 이슈 단위로 보완한다.

이름	담당 역할	주요 업무	담당 산출물·완료 기준
백진욱 (팀장)	아키텍처·핵심 개발·통합	일정과 이슈 관리, 백엔드 기본 구조, PostgreSQL 설계, 핵심 API, AI Adapter, 프론트·서버 통합, GitHub 리뷰, 배포와 발표 총괄	API·DB·AI 가 연결된 통합 빌드, main 브랜치 안정화, 최종 시연과 제출물
김하늘	프론트엔드 UI·UX	페이지별 CSS 정리, 공통 컴포넌트 적용, 모바일 반응형, 빈 상태·오류 상태, 사용성·접근성 개선, 화면별 시각 일관성 점검	핵심 페이지의 반응형 완료, 잘못된 점 없음, 버튼과 입력 요소의 정상 동작
장근우	기능 검증·콘텐츠·문서	Postman API 확인, 테스트·예시 데이터 작성, 문제·해설 품질 점검, 가이드 화면, README·설치 안내·시연 순서와 오류 재현 기록	테스트 체크리스트와 샘플 데이터, 실행 가능한 문서, 시연 시나리오·오류 보고서

14.3 GitHub 협업 규칙

- 기능마다 GitHub Issue 를 만들고 담당자와 완료 기준을 기록한다.
- main 에서 직접 작업하지 않고 feature/이름-기능 형식의 브랜치를 사용한다.
- 한 번에 큰 코드를 올리지 않고 화면·API·문서 단위로 작은 커밋을 만든다.
- Pull Request 에는 변경 내용·확인 방법·스크린샷을 첨부하고, 백진욱이 통합 테스트 후 main 에 병합한다.

15. 테스트 및 품질 관리

15.1 기능 테스트

시험 항목	통과 기준
학습 기록	생성·조회·수정·삭제 후 새로고침에도 데이터 일치
오답 이미지	허용 형식·용량 검증, 미리보기와 삭제 정상
문제 검토	객관식·OX·단답형 수정·승인 후 퀴즈에 동일 반영
퀴즈	정답 판정·해설·점수·재시도 상태 정확
통계	기록 합계와 그래프·활동일·히트맵 값 일치
API·DB	잘못된 요청은 4xx, 정상 요청은 DB 에 정확히 반영
UI	모바일·태블릿·데스크톱에서 겹침·잘림·가로 넘침 없음

15.2 자동·수동 품질 확인

- 프론트엔드: npm run lint, npm run build
- 백엔드: TypeScript 빌드, API 단위 테스트, Zod 입력 검증
- 데이터베이스: 마이그레이션 재실행, 외래키·삭제 정책 확인
- 수동 E2E: 기록 작성 → 오답 등록 → 문제 승인 → 퀴즈 → 통계 반영
- 팀원 교차 테스트: 개발자가 아닌 사람이 같은 기능을 다시 확인

16. 위험요소 및 대응

위험요소	영향	대응 방안
AI 오답 환각	잘못된 학습 유도	범위 제한, JSON 검증, 사용자 검토·승인, Mock 대체
이미지·OCR 오류	문제 텍스트 왜곡	MVP는 직접 입력·수정 중심, 이미지 원본 병행
저작권	문제집 원문 재배포 위험	개인 복습 용도, 외부 공유 금지, 변형 문제 생성
개인정보·비밀키	유출 위험	.env 제외, 예시 데이터 사용, 업로드 삭제 정책
외부 API 장애	문제 생성 중단	Mock Generator와 승인된 기존 문제로 시연
개발 범위 과다	완성도 저하	MVP 우선, OCR·랭킹·교사용 화면 분리
팀원 경험 부족	통합 지연	작은 이슈, 예제 제공, 페어 작업, 주 2회 통합

17. 기대 효과

관점	기대 효과
사용자	오답노트 작성 부담을 줄이고 취약 개념을 반복 복습하며 학습 동기를 유지
교육	계획·수행·검증·복습의 순환과 회상 연습을 결합해 자기주도 학습 강화
팀	React·TypeScript·Express·PostgreSQL·GitHub 협업의 실제 경험 축적
오픈소스	다른 교육 서비스가 재사용할 문제 스키마·판정 규칙·AI Adapter 제공

18. 향후 발전 방향

우선순위	발전 기능	설명
1	간격 반복	오답과 이해도에 따라 다음 복습일 추천
2	OCR 보조	문제 이미지에서 텍스트를 추출하고 사용자가 교정
3	난이도 조절	정답률·시도 횟수에 따라 다음 문제 난이도 조절
4	취약 개념 그래프	연결된 개념과 반복 오류를 시각화
5	교사·멘토 검수	문제 승인과 피드백을 공동으로 관리
6	PWA·모바일	설치형 웹 앱과 알림 기반 복습
7	로컬 AI·플러그인	AI 제공자 교체와 과목별 생성 규칙 확장
8	데이터 내보내기	학습 기록 JSON·CSV 와 상호운용 API

19. 최종 완료 기준

1. 사용자가 과목과 공부한 내용을 등록하고 서버에서 다시 조회할 수 있다.
2. 오답 이미지와 문제·정답·오답·틀린 이유를 등록하고 상세 보기에서 확인할 수 있다.
3. 취약 개념을 바탕으로 복습 문제 3~5 개를 생성하거나 Mock 문제로 동일 흐름을 시연할 수 있다.
4. 사용자가 생성 문제의 유형·보기·정답·해설을 검토·수정·승인할 수 있다.
5. 객관식·OX·단답형 문제를 풀고 즉시 채점과 해설을 확인할 수 있다.
6. 풀이 결과와 이해도가 학습 기록과 통계에 반영된다.
7. 반응형 화면에서 주요 기능이 겹치거나 잘리지 않는다.
8. GitHub README 만으로 프론트·백엔드·DB 개발 환경을 실행할 수 있다.
9. 비밀키와 개인정보가 저장소에 포함되지 않고 테스트 데이터만 공개된다.
10. 대회 시연 영상에서 전체 흐름이 안정적으로 재현된다.

최종 핵심 가치

학습 계획을 완료하는 서비스가 아니라, 학습 내용을 이해했는지 확인하고 다음 복습 행동까지 연결하는 서비스

부록 A. 제출 전 최종 체크리스트

영역	확인 항목	담당
코드	main 최신화, lint-build-API 테스트 통과	백진욱
UI	주요 해상도 점검, 빈·오류 상태, 디자인 일관성	김하늘
문서·QA	README, 샘플 데이터, 시연 순서, 기능 체크	장근우
보안	.env·비밀번호·개인정보·저작권 자료 제외	전원
발표	문제·해결·차별성·데모·향후 계획 5 분 구성	전원

reQuest · Review what you learned. Complete what you understand.